



UNIVERSITY OF AZAD JAMMU & KASHMIR

Department of Computer Science

OBJECT-ORIENTED PROGRAMMING

E-VOTING MANAGEMENT SYSTEM

C++ Based Object-Oriented Management System

Project Type	Course Project
Programming Language	C++
Academic Level	BS Computer Science — 2nd Semester
Programming Paradigm	Object-Oriented Programming

Project Documentation

1. Project Overview

The E-Voting Management System is a console-based C++ application developed to demonstrate the practical use of Object-Oriented Programming concepts. The system models a basic election workflow in which voters can be registered, candidates can be maintained, votes can be cast once per registered voter, and election results can be calculated and displayed.

This project is designed specifically as an academic OOP project. It focuses on software structure, classes, objects, inheritance, polymorphism, encapsulation, file handling, validation, and menu-driven interaction. It is not intended to replace a secure governmental election platform.

2. Abstract

The application uses four major classes: Person, Voter, Candidate, and ElectionSystem. Person provides common identity information, while Voter and Candidate inherit from it. ElectionSystem manages the complete application workflow and stores collections of voters and candidates.

The system stores information in two text files, voters.txt and candidates.txt. When the program starts, saved records are loaded. During registration or voting, updated records are written back to the files. A voter is identified through a 13-digit CNIC and must be at least 18 years old.

3. Problem Statement

Manual management of voter records and vote counting requires repeated data entry and checking. The aim of this project is to develop a small computerized model that organizes voter and candidate information and automates the basic voting workflow.

4. Project Objectives

- Apply Object-Oriented Programming concepts in a practical C++ project.
- Create reusable classes for people, voters, and candidates.
- Register voters with basic validation.
- Maintain candidate records and vote counts.
- Allow a registered voter to cast only one vote.
- Calculate total votes and candidate percentages.
- Store and reload information using file handling.
- Provide a clear menu-driven interface for demonstration.

5. System Features

Feature	Description
Voter Registration	Collects name, CNIC and age; checks format, age and duplicate CNIC.
Voter Display	Shows registered voters and whether each voter has voted.
Candidate Management	Adds candidates and stores candidate ID, name, party and votes.
Vote Casting	Checks voter registration and voting status before accepting a vote.
Search Voter	Finds a voter using the CNIC.
Election Results	Displays vote totals, percentages and the leading candidate.
File Storage	Persists voter and candidate data in text files.

6. Technologies and Requirements

Requirement	Details
Language	C++
IDE	Visual Studio Code / Code::Blocks / Dev-C++ / Visual Studio
Compiler	Standard C++ compiler supporting C++11 or later
Interface	Console / Terminal
Storage	Plain text files
Libraries	iostream, fstream, vector, string, iomanip, limits, stdexcept

7. OOP Concepts Used

Concept	Application in Project
Class	Person, Voter, Candidate and ElectionSystem define data and behavior.
Object	Actual Voter, Candidate and ElectionSystem objects are created during execution.
Encapsulation	Private members such as age, hasVoted, party and votes are accessed through methods.
Inheritance	Voter and Candidate inherit common name/CNIC information from Person.
Polymorphism	Person declares virtual display(); derived classes override it.
Constructors	Constructors initialize objects with default or supplied values.
Abstraction	ElectionSystem hides internal searching, storage and voting logic behind public methods.

8. Class Architecture and Working

The design separates common identity information from role-specific information. Person is the parent class. Voter and Candidate extend Person with their own data and behavior. ElectionSystem acts as the controller and keeps vectors of Voter and Candidate objects.

8.1 Person Class

Person contains name and cnic. These are protected so derived classes can use them. The class also defines a virtual display() function, enabling polymorphism.

8.2 Voter Class

Voter inherits from Person and adds age and hasVoted. The castVote() method changes the voting status from false to true. This status is used to stop a second vote from the same stored voter.

8.3 Candidate Class

Candidate inherits from Person and adds party and votes. The addVote() method increments the candidate's vote count whenever a valid vote is cast.

8.4 ElectionSystem Class

ElectionSystem contains the main application logic. It manages voter and candidate vectors and provides registration, searching, voting, result calculation, saving, loading and menu methods.

9. Detailed Working of the Program

Step 1 — Program Start: main() creates an ElectionSystem object. The constructor loads existing records from the text files. If no candidates exist, the program creates three sample candidates.

Step 2 — Voter Registration: The user enters name, CNIC and age. validCNIC() verifies that the CNIC contains exactly 13 digits. findVoter() checks for duplicate registration. Age must be 18 or above.

Step 3 — Candidate Selection: When casting a vote, the registered voter enters a candidate ID. findCandidate() searches the candidate vector.

Step 4 — Vote Validation: The system first verifies that the voter exists and has not voted before. If hasVoted is true, the vote is rejected.

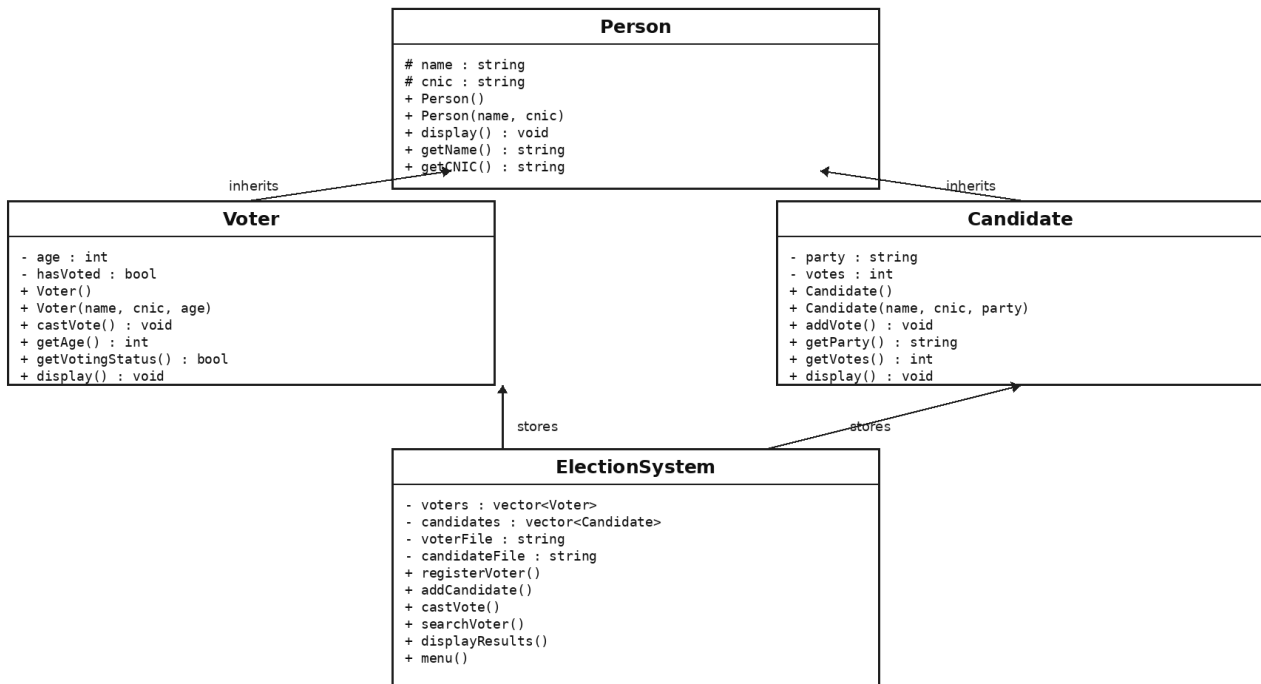
Step 5 — Vote Recording: addVote() increments the selected candidate's vote count and castVote() marks the voter as having voted. Both files are then saved.

Step 6 — Results: The system adds all candidate votes to calculate total votes and calculates each candidate's percentage. The highest vote count identifies the leading candidate.

10. UML Class Diagram

The following UML class diagram represents the main structure of the project. It shows inheritance between Person, Voter and Candidate, and the role of ElectionSystem in storing and managing these objects.

UML CLASS DIAGRAM — E-VOTING MANAGEMENT SYSTEM



UML Relationship Explanation

- Voter inherits Person: A voter is a person and reuses the common name and CNIC attributes.
- Candidate inherits Person: A candidate also reuses the common identity information.
- ElectionSystem stores Voter objects: The system maintains a vector containing registered voters.
- ElectionSystem stores Candidate objects: The system maintains a vector containing candidates.
- Polymorphic display(): Person provides a virtual display() function and both derived classes provide their own versions.

11. File Handling

The project uses two text files for persistence. This means that closing the program does not automatically remove the stored records.

File	Stored Data	Format
voters.txt	Name, CNIC, age, voting status	name cnic age status
candidates.txt	Name, candidate ID, party, vote count	name id party votes

The `saveVoters()` and `saveCandidates()` methods use `ofstream` to write records. The `loadVoters()` and `loadCandidates()` methods use `ifstream` to read records when the application starts. The pipe symbol (`|`) separates fields.

12. Validation and Error Handling

- CNIC must contain exactly 13 digits.

- Voter age must be 18 or above.
- Duplicate voter CNIC is rejected.
- Duplicate candidate ID is rejected.
- Unregistered voters cannot cast votes.
- A voter marked as already voted cannot vote again.
- Invalid numeric input is handled by `getInteger()`.
- Invalid saved records are ignored instead of stopping the complete program.

13. Program Flow and Sample Execution

The complete workflow starts with loading saved data, continues through the menu-driven operations, and returns to the menu after each operation.

START
Load voter and candidate files
Display Main Menu
Register / Display / Add Candidate / Cast Vote / Search / Results
Save changed data
Return to Menu or Exit
END

13.1 Example — Voter Registration

Enter full name: Laiba Ahmed
Enter 13-digit CNIC: 1234567890123
Enter age: 20
Voter registered successfully!

13.2 Example — Successful Vote

Enter your 13-digit CNIC: 1234567890123
Enter Candidate ID: C001
VOTE CAST SUCCESSFULLY!

13.3 Example — Duplicate Vote

If the same voter tries to vote again, the system checks hasVoted and displays: You have already cast your vote.

14. Testing Summary

Test Case	Expected Behavior
Valid registration	Voter is stored successfully.
Invalid CNIC	Registration is rejected.
Age below 18	Registration is rejected.
Duplicate CNIC	Existing voter is not duplicated.
Valid vote	Candidate vote count increases and voter is marked voted.
Duplicate vote	Second vote is rejected.
Invalid candidate ID	Vote is rejected.
Restart application	Previously saved records are loaded.

15. Limitations

- This is a console-based academic prototype.
- Text files are not encrypted.
- CNIC validation checks only the format, not real identity.

- There is no administrator authentication.
- The system is not a secure real-world election platform.

16. Future Enhancements

- Add administrator and voter login roles.
- Replace text files with SQLite or MySQL.
- Encrypt sensitive information.
- Add a graphical user interface.
- Add audit logs and stronger access control.
- Add secure backup and recovery.
- Add stronger identity verification for a real-world design.

17. Conclusion

The E-Voting Management System demonstrates how Object-Oriented Programming can be applied to a practical C++ application. The project connects theoretical OOP concepts with a complete menu-driven workflow involving registration, candidate management, voting, result calculation and file persistence. The class structure also provides a strong foundation for future improvements such as databases, GUI development and stronger security controls.